DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CSA0607 - Design and Analysis of Algorithms

ASSIGNMENT 1 - CO1

Design, Analyse and Validate an Efficient Search Strategy for a Large-Scale Student Record System

(Total Marks - 50)

Field	Details
Student name	A. Mohith Krishna
Registration number	192425199
Course code	CSA0607
Course name	Design and Analysis of Algorithms
Submission date	15-10-2026

Contents

- Task 1 - Design the Algorithms (5 marks)
- Task 2 - Identify the Basic Operation (2 marks)
- Task 3 - Time Complexity, Derived Mathematically (6 marks)
- Task 4 - Best, Average and Worst Cases (4 marks)
- Task 5 - Asymptotic Analysis (3 marks)
- Task 6 - Formulate the Recurrence Relation (3 marks)
- Task 7 - Solve the Recurrence Relation (4 marks)
- Task 8 - Implement and Test (4 marks)
- Task 9 - Record Experimental Performance (3 marks)
- Task 10 - Compare Theory with Experiments (4 marks)
- Task 11 - Recommend and Defend the Final Design (4 marks)
- Task 12 - Day-4 Design Challenge (8 marks)
- Summary and conclusion
- Appendix - Reproduction notes

Summary of the work

Three strategies for finding a student registration number were designed, analysed mathematically, implemented in Python and measured: **linear search** (iterative, no preprocessing), **recursive binary search** (preprocessing: sorting) and a **hash table with separate chaining** (preprocessing: table build). They were tested on 1,000, 10,000, 100,000 and 1,000,000 synthetic 9-digit registration numbers stored in random order.

At 1,000,000 records the measured average search time was **18.8 ms** for linear search, **2.45 microseconds** for recursive binary search and **0.12 microseconds** for the hash table. Average comparison counts followed theory: about n/2 for linear search, log2(n) - 1 for binary search and 1 + alpha/2 (about 1.3) for the hash table. Binary search on a sorted array is recommended for the original scenario (Task 11). For the Day-4 workload of 50,000 searches per hour with one update per day, the design is revised to use a hash index rebuilt once per day (Task 12); the report states honestly that binary search would also cope with that load.

Task 1 - Design the Algorithms (5 Marks)

Problem. Given n student records, each identified by a unique 9-digit registration number, decide whether a key K exists and return the position of its record. Records are held in memory in the order they were created, so the array is not sorted. The database grows from about 1,000 to 1,000,000 records or more.

Three strategies that differ in design technique, preprocessing and cost behaviour were chosen. One is iterative only (A), one is recursive (B) and one is iterative with a data-structure build (C).

Table 1.1 Summary of the three approaches

Approach	Design strategy	Form	Preprocessing	Data structure
A. Linear search	Brute force	Iterative	None	Unsorted array
B. Binary search	Decrease and conquer (by half)	Recursive	Sort: Theta(n log n)	Sorted array
C. Hash-based search	Transform and conquer (space-time trade-off)	Iterative	Build table: Theta(n)	m buckets with chaining

Approach A - Linear search (iterative, brute force)

Basic idea. Examine the records one after another from the first position until the key is found or the end of the array is reached. No assumption is made about the order of the data.

```
ALGORITHM LinearSearch(A[0..n-1], K)
  for i <- 0 to n-1 do
    if A[i] = K then          // basic operation
      return i
  return -1
```

Flowchart: Linear search (iterative)

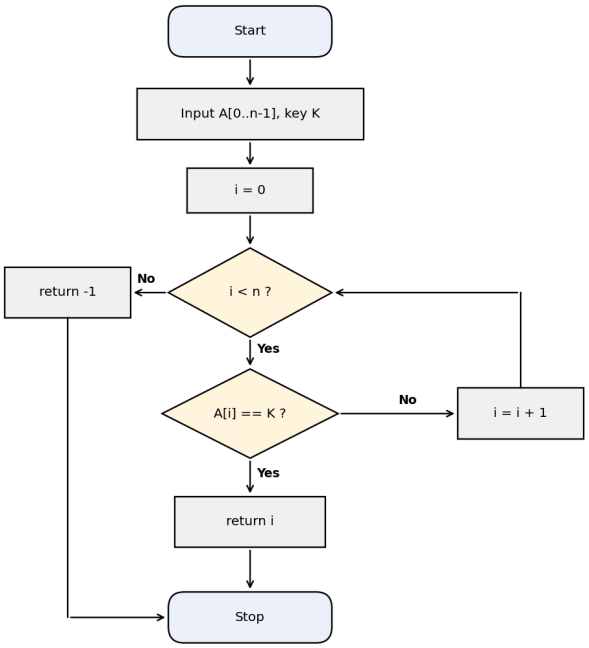


Figure 1.1 Flowchart of linear search

Preprocessing: none. **Reason for selection:** it is the natural baseline. It needs no setup, so it is the best choice for a tiny database or a one-off query, and it gives the reference against which the benefit of preprocessing is measured.

Approach B - Binary search (recursive, decrease and conquer)

Basic idea. If the registration numbers are sorted, compare K with the middle element. A match ends the search. Otherwise the half that cannot contain K is discarded and the same procedure is applied to the remaining half, so each call halves the problem.

```
ALGORITHM BinarySearch(A[low..high], K)      // A sorted ascending
  if low > high then return -1               // base case: empty range
  mid <- floor((low + high) / 2)
  if A[mid] = K then return mid              // basic operation (one probe)
  else if K < A[mid] then
    return BinarySearch(A, K, low, mid - 1) // left half
  else
    return BinarySearch(A, K, mid + 1, high)// right half

PREPROCESSING: A <- Sort(records)   // Theta(n log n)
FIRST CALL:    BinarySearch(A, K, 0, n-1)
```

Flowchart: Binary search (recursive)

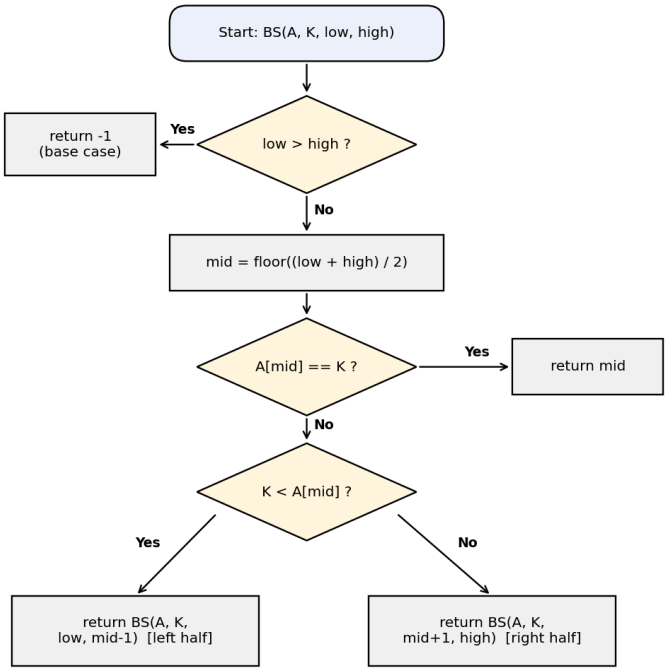


Figure 1.2 Flowchart of recursive binary search

Preprocessing: the records must be sorted first; this cost is included in every later analysis. An iterative loop version was also coded, but only to measure the overhead of recursion; it is not counted as a separate approach. **Reason for selection:** registration numbers are totally ordered integers, so sorting is possible, and logarithmic search time is the classic remedy for large databases. It is also the recursive algorithm analysed in Tasks 6 and 7.

Approach C - Hash-based search (hash table with chaining, iterative)

Basic idea. Instead of comparing keys repeatedly, compute where a key must be stored. The hash function h(K) = K mod m maps a registration number to one of m buckets, where m is a prime close to 1.5n (load factor alpha = n/m of about 0.67). Records that collide are chained in the same bucket. A search computes h(K) and scans only that short chain.

```
ALGORITHM BuildHashTable(records[0..n-1])
  m <- smallest prime >= 1.5 n + 1
  T[0..m-1] <- empty lists
```

```
  for each k in records do
    append k to T[k mod m]          // n insertions

ALGORITHM HashSearch(T, m, K)
  h <- K mod m
  for each x in T[h] do
    if x = K then return FOUND      // basic operation
  return NOT FOUND
```

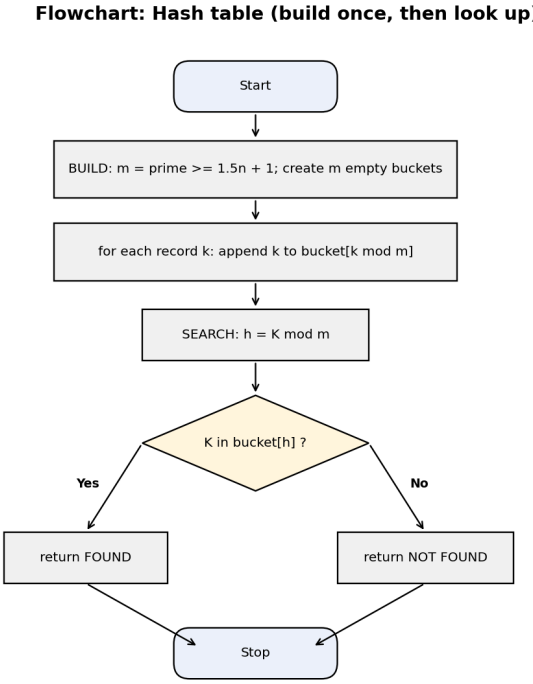


Figure 1.3 Flowchart of hash table build and lookup

Preprocessing: insert all n records once (Theta(n)). **Reason for selection:** the problem is exact-match lookup on a unique key, which is what hashing is designed for. It trades extra memory (about 1.5 slots per record) and a build cost for an expected constant search time, which matters when the number of searches is large (Task 12).

Diversity of the set. A imposes no structure, B imposes order (sorting) and C imposes address calculation (hashing). A and C are iterative and B is recursive, so every requirement of the brief is met.

Task 2 - Identify the Basic Operation (2 Marks)

The basic operation is the operation executed most often in the innermost part of an algorithm, so its count determines the running time up to a constant factor. In all three algorithms it is a key comparison, but each lives in a different place and grows differently.

Table 2.1 Basic operations

Algorithm	Basic operation	Where it occurs	Count as n grows
Linear search	Compare A[i] with K	Body of the for loop	Proportional to n (average about n/2, worst n)
Binary search	Compare K with A[mid]	Once per recursive call	Grows by 1 each time n doubles: floor(log2 n) + 1 worst case
Hash search	Compare x with K inside the chain of bucket T[h]	Loop over one bucket	Independent of n; depends on load factor alpha = n/m

Linear search. Each iteration does an index increment, a bound check and one comparison with the key. The comparison runs once per iteration, so the number of comparisons equals the number of records examined. The other statements run the same number of times and only change the constant factor. If the database doubles, the expected comparisons double.

Binary search. Each call computes mid (constant work), compares K with A[mid] and makes at most one recursive call. The number of comparisons therefore equals the number of calls, which is the recursion depth. Because the range halves every call, doubling n adds only one call. In the code one probe is counted as one comparison (the three-way decision equal / less / greater is one basic step).

Hash search. Computing K mod m is one arithmetic operation per search and does not depend on n. What varies with the data is the scan of the chain, whose length is the number of records that collided with K, so the key comparison inside that scan is chosen as the basic operation. Because m grows in proportion to n, the average chain length stays near alpha and the comparison count stays nearly constant.

Preprocessing operations. For binary search the basic operation of the sort is the comparison of two registration numbers (about n log2 n of them). For the hash table it is one insertion (hash computation plus append), executed n times. Linear search has no preprocessing.

Task 3 - Time Complexity, Derived Mathematically (6 Marks)

Input size n is the number of records. C(n) is the number of basic operations in one search, and running time is T(n) = c * C(n) + c0, where c is the cost of one basic step and c0 a constant set-up cost. For average successful searches, the key is assumed equally likely (probability 1/n) to be at each position.

A. Linear search

If the key is at position i (1-based) the loop performs exactly i comparisons.

Successful search at position i: C(i) = i

Average (successful) = (1/n) * SUM(i = 1..n) i = (1/n) * n(n+1)/2 = (n+1)/2

Unsuccessful search: all n records examined, C = n. Worst case: C(n) = n

If a fraction p of queries are for existing keys, C_avg = p(n+1)/2 + (1-p)n. In every case C(n) is linear in n, so T(n) = a n + b with a > 0. Preprocessing cost is 0, so q searches cost q(n+1)/2 on average and q n in the worst case. For n = 1,000,000 the average is about 500,000 comparisons per search.

B. Binary search with sorting

Searching. After the j-th probe at most floor(n / 2^j) elements remain, and the search must stop when the range is empty:

floor(n / 2^j) = 0 <=> 2^j > n <=> j >= floor(log2 n) + 1

C_worst(n) = floor(log2 n) + 1

Average successful case. Take n = 2^k - 1. The probes form a balanced decision tree in which 2^(j-1) elements are found at probe number j, so

SUM(j = 1..k) j * 2^(j-1) = (k-1) 2^k + 1 (check k = 3: 1 + 4 + 12 = 17 = 2*8 + 1)

C_avg(n) = [(k-1) 2^k + 1] / (2^k - 1) ~ k - 1 = log2(n+1) - 1

For n = 1,000,000 this gives 20 probes in the worst case and about 19 on average. **Preprocessing.** Merge sort satisfies S(n) = 2 S(n/2) + (n - 1) comparisons with S(1) = 0, whose solution is S(n) about n log2 n (Master Theorem case 2). For n = 1,000,000, n log2 n = 1,000,000 x 19.93 = about 1.99 x 10^7 comparisons.

T_total(n, q) = n log2 n + q log2 n = Theta(n log n + q log n)

C. Hash table with chaining

Table size m is about 1.5 n, so the load factor is alpha = n/m = 0.67. Under simple uniform hashing the chain in a bucket has expected length alpha.

Unsuccessful search: scans a whole chain, C = alpha = 0.67 comparisons on average

Successful search: C = 1 + alpha/2 = 1.33 comparisons on average

T_search = c1 (hash computation) + c2 * C = Theta(1 + alpha) = Theta(1), independent of n

Worst case. All keys fall in one bucket, so C = n and the search is Theta(n). With a prime m and registration numbers that behave randomly modulo m this does not happen in practice (longest chain measured: 4, 6, 6 and 4 for the four sizes), but Task 4 shows that deliberately chosen keys do trigger it.

Preprocessing. Building needs n insertions of O(1) each plus allocation of m = 1.5 n buckets:

T_build(n) = n c_ins + m c_alloc = Theta(n) T_total(n, q) = Theta(n + q)

Table 3.1 Theoretical operation counts at n = 10^6

Algorithm	Preprocessing (operations)	Per search (average)	Total for q searches
Linear	0	about 5.0 x 10^5 comparisons	5.0 x 10^5 q
Binary + sort	about 2.0 x 10^7 comparisons	about 19 comparisons	2.0 x 10^7 + 19 q
Hash table	about 1.0 x 10^6 insertions + 1.5 x 10^6 bucket allocations	about 1.3 comparisons	about 2.5 x 10^6 + 1.3 q

Break-even (theory, operation counts). Sorting pays for itself against linear search when n log2 n + 19 q < q n/2, i.e. q > n log2 n / (n/2 - 19), which is about 2 log2 n = 40 searches for n = 10^6. The hash table pays for its build earlier still in operation count, because its preprocessing is Theta(n) rather than Theta(n log n). Task 10 compares these predictions with measured times.

Task 4 - Best, Average and Worst Cases (4 Marks)

Linear search. *Best case:* the key is the first record; the first comparison succeeds, C = 1 = Theta(1). *Average case:* the key is equally likely to be anywhere, so about half the array is scanned, C = (n+1)/2 (it rises towards n if the key may be absent). *Worst case:* the key is last or absent, so every record is compared, C = n. The differences arise only from where the key sits relative to the start of an unsorted array, because nothing about the data lets the algorithm skip a record.

Binary search. *Best case:* the key is at the first mid position, one probe, C = 1. *Average case:* half of all keys lie at the deepest level of the decision tree, a quarter at the next, and so on, so the expected depth is about log2 n - 1. *Worst case:* the key is absent or is found only when the range has shrunk to one element, giving floor(log2 n) + 1 probes. The count is bounded by the height of the halving process, so even the worst case is only a probe or two above the average. The array must be sorted, otherwise discarding half the range is invalid.

Hash search. *Best case:* the key is alone in its bucket, C = 1 (or C = 0 for an absent key whose bucket is empty). *Average case:* with alpha = 0.67 the expected comparisons are 1 + alpha/2 = 1.33 for present keys and alpha = 0.67 for absent keys. *Worst case:* all n keys collide in one bucket and the lookup becomes a linear scan, C = n = Theta(n). This needs a poor hash function or adversarial keys, which is why hashing has a good expected bound but a weaker guaranteed bound.

Table 4.1 Operation counts by case (derived)

Case	Linear (comparisons)	Binary (comparisons)	Hash (comparisons)
Best	1 (key first)	1 (key at first mid)	1 (alone in bucket)
Average (key present)	(n+1)/2	about log2 n - 1	about 1 + alpha/2 = 1.33
Worst	n (last or absent)	floor(log2 n) + 1	n (all collide); in practice 4-6

Measured behaviour at n = 1,000,000

Table 4.2 Measured comparison counts at n = 1,000,000 (first / middle / last refer to the unsorted storage order)

Query	Linear	Binary (recursive)	Hash
First record	1	20	1
Middle record	500,001	19	1
Last record	1,000,000	19	2
Absent key	1,000,000	19	2
Average of 15 random present keys	629,546.5	19.7	1.27

Linear search needs 1 comparison for the first record and 10^6 for the last or an absent key. Binary search needs 19-20 probes whatever the key, showing how little best and worst differ. The hash table needs 1-2 comparisons in every case here. The linear average (629,547) is above n/2 only because it is the mean of just 15 random keys, discussed in Task 10.

Demonstrating the hash worst case

To show that the Theta(n) hash worst case is real, a second experiment (extra_experiments.py, Task 8) builds a table of 10,000 keys that are all multiples of m. Every key lands in bucket 0, the longest chain is 10,000, and finding the last key needs 10,000 comparisons and about 196 microseconds, against about 0.12 microseconds for random keys.

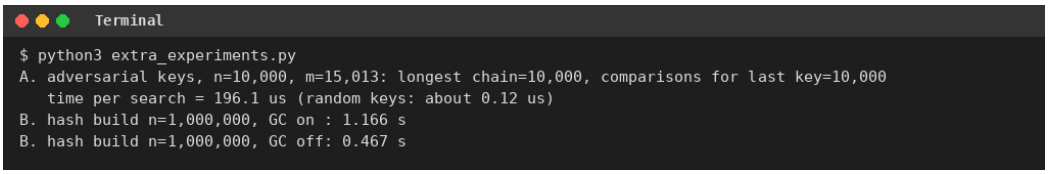


Figure 4.1 Output of extra_experiments.py (adversarial keys, and hash build with and without garbage collection)

Task 5 - Asymptotic Analysis (3 Marks)

Definitions: f(n) = O(g(n)) if f(n) <= c g(n) for all n >= n0; f(n) = Omega(g(n)) if f(n) >= c g(n) for all n >= n0; f(n) = Theta(g(n)) if both hold.

Table 5.1 Asymptotic summary

Algorithm	Best	Average	Worst	Preprocessing	Overall (q searches)
Linear	Theta(1)	Theta(n)	Theta(n)	none	Theta(q n)
Binary	Theta(1)	Theta(log n)	Theta(log n)	Theta(n log n)	Theta(n log n + q log n)
Hash	Theta(1)	Theta(1) expected	Theta(n) (O(1) in practice)	Theta(n)	Theta(n + q) expected

Linear search. C_avg = (n+1)/2 satisfies n/2 <= (n+1)/2 <= n for n >= 1, so it is Theta(n) with c1 = 1/2, c2 = 1, n0 = 1. The worst case is exactly n, so the running time is O(n) overall and Omega(1) over all inputs; it is Theta(n) for the average and worst cases but not Theta of one function across all inputs.

Binary search. C_worst = floor(log2 n) + 1 and log2 n <= floor(log2 n) + 1 <= 2 log2 n for n >= 2, so it is Theta(log n).

Hash search. The expected cost 1 + alpha/2 with alpha fixed near 0.67 is bounded above and below by constants, so it is Theta(1) in expectation, while the worst case is O(n).

Order of growth. log n = O(n) but n is not O(log n), since lim (log2 n)/n = 0 as n tends to infinity (L'Hopital: (1/(n ln 2)) / 1 tends to 0). So 1 << log n << n << n log n.

Table 5.2 Predicted growth for the four database sizes

Records n	Linear (avg comparisons)	Binary (worst probes)	Hash (expected)	Sorting cost n log2 n
1,000	500	10	about 1.3	9,966
10,000	5,000	14	about 1.3	132,877
100,000	50,000	17	about 1.3	1,660,964
1,000,000	500,000	20	about 1.3	19,931,569

Behaviour at one million records and beyond

When the database grows by a factor of 1,000 (1,000 to 1,000,000 records) linear search does 1,000 times more work per query, binary search does only about twice as much (10 to 20 probes) and the hash table does the same amount. A search that is imperceptible at 1,000 records (about 10 microseconds measured) takes about 19 milliseconds at 10^6 records, which a front desk serving many requests would notice. Binary search and hashing stay practical at any size the university is likely to reach: doubling the database costs binary search one more probe and the hash table nothing. The price is preprocessing, which grows as n log n for binary search and as n for hashing, and is paid only when the data changes.

Task 6 - Formulate the Recurrence Relation (3 Marks)

The recursive algorithm is recursive binary search (Task 1, Approach B). The recurrence is derived directly from its code.

```
BinarySearch(A[low..high], K)
   if low > high then return -1              // base case
   mid <- floor((low + high) / 2)
   if A[mid] = K then return mid
   else if K < A[mid] then return BinarySearch(A, K, low, mid - 1)
   else                    return BinarySearch(A, K, mid + 1, high)
```

Table 6.1 Components of the recurrence

Component	Identification from the algorithm
Input size	n = high - low + 1, the number of records still in the range (initially the whole database)
Size of the subproblem	mid = floor((low + high)/2). The left range low..mid-1 has floor((n-1)/2) elements; the right range mid+1..high has floor(n/2). The larger, floor(n/2), is used for the worst case
Number of recursive calls	Exactly one (left or right, never both), so a = 1
Work outside the recursive call	Compute mid and make one or two comparisons: constant work c, so f(n) = Theta(1). Counted as C(n), one probe = 1
Base case	low > high, i.e. n = 0: the range is empty, the key is absent and the call returns -1 with no probe. T(0) = c0, C(0) = 0

A call on a range of n >= 1 elements does constant work and, if the key was not at mid, makes a single call on at most floor(n/2) elements. Therefore:

$T(n) = T(\text{floor}(n/2)) + c \quad \text{for } n >= 1, \quad T(0) = c0$

$C(n) = C(\text{floor}(n/2)) + 1 \quad \text{for } n >= 1, \quad C(0) = 0 \quad \text{(in probes)}$

For the Master Theorem the usual simplification $T(n) = T(n/2) + \text{Theta}(1)$ is used, with a = 1, b = 2, $f(n) = \text{Theta}(1)$. Contrast this with $T(n) = 2T(n/2) + n$ (merge sort): here only one half is solved, and that is what makes the algorithm logarithmic.

Task 7 - Solve the Recurrence Relation (4 Marks)

Recurrence from Task 6: C(n) = C(floor(n/2)) + 1 for n >= 1, C(0) = 0; equivalently T(n) = T(n/2) + c.

Method 1 - Iteration (backward substitution)

With a single recursive term, repeated expansion shows a simple pattern. Take n = 2^k to remove the floor:

T(n) = T(n/2) + c = T(n/4) + 2c = T(n/8) + 3c = ... = T(n/2^i) + i c

The expansion stops at the base case n/2^i = 1, i.e. i = log2 n. With T(1) = T(0) + c = c0 + c:

T(n) = T(1) + c log2 n = c log2 n + (c0 + c) = Theta(log n)

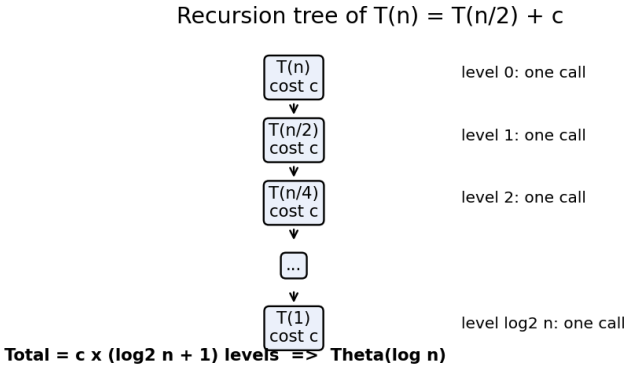


Figure 7.1 Recursion tree: one call per level, log2 n + 1 levels, constant cost c per level

Method 2 - Substitution (proof for all n, not only powers of 2)

Claim: C(n) = floor(log2 n) + 1 for all n >= 1.

Basis: C(1) = C(0) + 1 = 1 = floor(log2 1) + 1. *Inductive step:* assume the claim for all values below n (n >= 2). Then

C(n) = C(floor(n/2)) + 1 = [floor(log2 floor(n/2)) + 1] + 1 = [floor(log2 n) - 1] + 2 = floor(log2 n) + 1

using floor(log2 floor(n/2)) = floor(log2 n) - 1 for n >= 2. The claim holds, so the exact worst-case probe count is floor(log2 n) + 1, which is Theta(log n) and matches Task 3. For n = 10^6 it gives 20, which the experiment reproduces (Table 4.2).

Method 3 - Master Theorem (check)

T(n) = a T(n/b) + f(n) with a = 1, b = 2, f(n) = c = Theta(1).

n^(log_b a) = n^(log_2 1) = n^0 = 1, f(n) = Theta(1) = Theta(n^(log_b a)) => case 2

T(n) = Theta(n^(log_b a) log n) = Theta(log n)

Interpretation. Three independent methods give Theta(log n). The recursion depth is about log2 n: at most 10 for 1,000 records and at most 20 for 1,000,000 (measured 19-20). The depth is tiny, so the call stack cannot be exhausted (Python's default limit of about 1,000 frames is never approached) and recursion adds only a small constant per call. Together with the sorting cost from Task 3, binary search totals Theta(n log n + q log n).

Task 8 - Implement and Test (4 Marks)

All algorithms are implemented in Python 3 (tested with 3.12). Each algorithm has a plain version for timing and a counted version for recording comparisons; the counted version differs only by the counter increment, so it follows the same logic as the analysed pseudocode.

Table 8.1 Source files

File	Purpose
search_algorithms.py	linear_search, binary_search_recursive (plus iterative twin), HashTable class, sort_records
run_experiments.py	Correctness tests, dataset generation, timing and comparison counting; writes results.csv and results.json
extra_experiments.py	Hash worst-case demonstration and hash build with the garbage collector on and off
make_figures.py	Graphs, flowcharts and code/terminal screenshots

Test procedure (reproducible)

- **Datasets:** for each n in {1,000; 10,000; 100,000; 1,000,000}, n unique 9-digit registration numbers are drawn from 190,000,000-194,999,999 with random.sample and a fixed seed (2026 + n). Records are kept in random (unsorted) order.
- **Equivalent data:** the same list is given to all algorithms. Linear search scans it, a sorted copy is produced from it for binary search, and the hash table is built from it.
- **Queries per size:** first, middle and last record of the stored list; one absent key (189,999,999); and random present keys (300, 100, 40 and 15 for the four sizes; fewer for large n because linear search is slow).
- **Correctness first:** before any timing, every algorithm is checked to return the correct found / not-found answer on every query.
- **Time:** mean wall-clock time per search with time.perf_counter. Fast searches are repeated 200 times to beat timer resolution; linear search runs once per query. Sorting and hash build are timed separately.
- **Comparisons:** counted by the instrumented versions. Same machine, same process and same data for all algorithms.

Table 8.2 Test cases

No.	Test case	Expected result	Result
1	Empty database (n = 0), any key	Not found	Passed (all algorithms)
2	n = 1, 2, 3, 10, 257, 1000: first 50 stored keys	Found at a valid index	Passed
3	Key below smallest and above largest	Not found	Passed
4	Key 192,424,311 (may or may not be in the random data)	Same answer as Python's `in`	Passed
5	First, middle, last element of the unsorted list at all four sizes	Found	Passed
6	Random present keys at all four sizes	Found	Passed
7	Absent key at all four sizes	Not found	Passed

Each test also confirms that the index returned by binary search points at the key, and that recursive binary search, iterative binary search, linear search and the hash table all agree with Python's built-in membership test.

```
Terminal
$ python3 run_experiments.py
Correctness tests: PASSED (n = 0,1,2,3,10,257,1000; present and absent keys)
n=    1,000  sort=0.0001s  hash build=0.0001s  linear avg=9.8us  bin(rec) avg=1.07us  hash avg=0.12us
n=   10,000  sort=0.0015s  hash build=0.0014s  linear avg=110.7us  bin(rec) avg=1.57us  hash avg=0.12us
n=  100,000  sort=0.0219s  hash build=0.0210s  linear avg=1575.1us  bin(rec) avg=3.15us  hash avg=0.19us
n=1,000,000  sort=0.2919s  hash build=1.1768s  linear avg=18846.3us  bin(rec) avg=2.45us  hash avg=0.12us
```

Figure 8.1 Terminal output of run_experiments.py (real console output)

Source code

search_algorithms.py (page 1/3)

```python
"""
CSA0607 - Design and Analysis of Algorithms
CO1 Assignment: Search strategies for a student record system

Three strategies for finding a registration number:
  1. Linear search  - iterative, no preprocessing
  2. Binary search  - recursive (iterative twin kept for comparison), needs sorted data
  3. Hash search    - hash table with separate chaining, needs a table build

Every algorithm has a plain form (used for timing) and a counted form that
also returns the number of key comparisons (used for operation counts).
"""

# ------------------------------------------------------------------ 1. LINEAR
def linear_search(records, key):
    for i in range(len(records)):
        if records[i] == key:
            return i
    return -1


def linear_search_counted(records, key):
    comparisons = 0
    for i in range(len(records)):
        comparisons += 1
        if records[i] == key:
            return i, comparisons
    return -1, comparisons


# ------------------------------------------------------------------ 2. BINARY
def binary_search_recursive(records, key, low, high):
    if low > high:                      # base case: empty range
        return -1
    mid = (low + high) // 2
    if records[mid] == key:
        return mid
    if key < records[mid]:
        return binary_search_recursive(records, key, low, mid - 1)
    return binary_search_recursive(records, key, mid + 1, high)


def binary_search_recursive_counted(records, key, low, high, counter):
    if low > high:
        return -1
    mid = (low + high) // 2
    counter[0] += 1                     # one probe = one comparison
    if records[mid] == key:
        return mid
    if key < records[mid]:
```

Figure 8.2 search_algorithms.py, page 1 of 3

search_algorithms.py (page 2/3)

```
51          return binary_search_recursive_counted(records, key, low, mid - 1, counter)
52      return binary_search_recursive_counted(records, key, mid + 1, high, counter)
53
54
55  def binary_search_iterative(records, key):
56      low, high = 0, len(records) - 1
57      while low <= high:
58          mid = (low + high) // 2
59          if records[mid] == key:
60              return mid
61          if key < records[mid]:
62              high = mid - 1
63          else:
64              low = mid + 1
65      return -1
66
67
68  def sort_records(records):
69      """Preprocessing for binary search: Theta(n log n)."""
70      return sorted(records)
71
72
73  # ------------------------------------------------------------------ 3. HASH
74  class HashTable:
75      """Separate chaining, h(k) = k mod m, m = prime >= 1.5n + 1."""
76
77      @staticmethod
78      def _next_prime(x):
79          def is_prime(v):
80              if v < 2:
81                  return False
82              if v % 2 == 0:
83                  return v == 2
84              d = 3
85              while d * d <= v:
86                  if v % d == 0:
87                      return False
88                  d += 2
89              return True
90          while not is_prime(x):
91              x += 1
92          return x
93
94      def __init__(self, records):
95          n = len(records)
96          self.m = self._next_prime(int(1.5 * n) + 1)
97          self.table = [None] * self.m
98          for k in records:               # preprocessing: n insertions
99              h = k % self.m
100             if self.table[h] is None:
```

Figure 8.3 search_algorithms.py, page 2 of 3

search_algorithms.py (page 3/3)

```
                self.table[h] = [k]
            else:
                self.table[h].append(k)

    def search(self, key):
        bucket = self.table[key % self.m]
        if bucket is None:
            return False
        for k in bucket:
            if k == key:
                return True
        return False

    def search_counted(self, key):
        bucket = self.table[key % self.m]
        comparisons = 0
        if bucket is None:
            return False, comparisons
        for k in bucket:
            comparisons += 1
            if k == key:
                return True, comparisons
        return False, comparisons

```

Figure 8.4 search_algorithms.py, page 3 of 3

run_experiments.py (page 1/3)

```python
"""
Experiment driver. Usage: python3 run_experiments.py  -> results.json, results.csv
Same unsorted list of unique 9-digit registration numbers is given to every algorithm.
"""
import csv, json, os, random, sys, time
import search_algorithms as sa

sys.setrecursionlimit(10000)
SEED = 2026
SIZES = [1_000, 10_000, 100_000, 1_000_000]
HERE = os.path.dirname(os.path.abspath(__file__))
LO, HI = 190_000_000, 195_000_000


def timed(fn, queries):
    t0 = time.perf_counter()
    for q in queries:
        fn(q)
    return (time.perf_counter() - t0) / len(queries)


def correctness_tests():
    rnd = random.Random(1)
    for n in (0, 1, 2, 3, 10, 257, 1000):
        data = rnd.sample(range(LO, HI), n)
        srt = sa.sort_records(data)
        ht = sa.HashTable(data)
        probes = data[:50] + [LO - 1, HI + 1, 192_424_311]
        for k in probes:
            truth = k in data
            assert (sa.linear_search(data, k) != -1) == truth
            r = sa.binary_search_recursive(srt, k, 0, len(srt) - 1)
            assert (r != -1) == truth and (r == -1 or srt[r] == k)
            r2 = sa.binary_search_iterative(srt, k)
            assert (r2 != -1) == truth
            assert ht.search(k) == truth
    return True


def run():
    assert correctness_tests()
    print("Correctness tests: PASSED (n = 0,1,2,3,10,257,1000; present and absent keys)")
    results = []
    for n in SIZES:
        rnd = random.Random(SEED + n)
        data = rnd.sample(range(LO, HI), n)
        absent = LO - 1
        first, mid, last = data[0], data[n // 2], data[-1]
        qn = {1000: 300, 10000: 100, 100000: 40, 1000000: 15}[n]
        rand_keys = [data[rnd.randrange(n)] for _ in range(qn)]
```

Figure 8.5 run_experiments.py, page 1 of 3

run_experiments.py (page 2/3)

```
51
52          t0 = time.perf_counter(); srt = sa.sort_records(data); t_sort = time.perf_counter() - t0
53          t0 = time.perf_counter(); ht = sa.HashTable(data);      t_hash = time.perf_counter() - t0
54          hi = n - 1
55          rec = lambda k: sa.binary_search_recursive(srt, k, 0, hi)
56          itr = lambda k: sa.binary_search_iterative(srt, k)
57          lin = lambda k: sa.linear_search(data, k)
58          hsh = ht.search
59
60          for k in [first, mid, last, absent] + rand_keys:
61              truth = k != absent
62              assert (lin(k) != -1) == truth and (rec(k) != -1) == truth
63              assert (itr(k) != -1) == truth and hsh(k) == truth
64
65          def lin_c(k): return sa.linear_search_counted(data, k)[1]
66          def rec_c(k):
67              c = [0]; sa.binary_search_recursive_counted(srt, k, 0, hi, c); return c[0]
68          def hsh_c(k): return ht.search_counted(k)[1]
69          def avg(f, ks): return sum(f(k) for k in ks) / len(ks)
70
71          row = {"n": n, "sort_time_s": t_sort, "hash_build_time_s": t_hash,
72                 "hash_table_size": ht.m,
73                 "hash_max_chain": max(len(b) for b in ht.table if b),
74                 "hash_avg_chain_nonempty": sum(len(b) for b in ht.table if b) / sum(1 for b in ht.table if
75          for name, fn, cf in (("linear", lin, lin_c), ("binary_rec", rec, rec_c),
76                               ("binary_iter", itr, None), ("hash", hsh, hsh_c)):
77              rep = 1 if name == "linear" else 200
78              def timeit(qs): return timed(fn, qs * rep)
79              row[f"{name}_time_first_s"] = timeit([first])
80              row[f"{name}_time_mid_s"] = timeit([mid])
81              row[f"{name}_time_last_s"] = timeit([last])
82              row[f"{name}_time_absent_s"] = timeit([absent])
83              row[f"{name}_time_avg_s"] = timeit(rand_keys)
84              if cf:
85                  row[f"{name}_cmp_first"] = cf(first)
86                  row[f"{name}_cmp_mid"] = cf(mid)
87                  row[f"{name}_cmp_last"] = cf(last)
88                  row[f"{name}_cmp_absent"] = cf(absent)
89                  row[f"{name}_cmp_avg"] = avg(cf, rand_keys)
90          results.append(row)
91          print(f"n={n:>9,}  sort={t_sort:.4f}s  hash build={t_hash:.4f}s  "
92                f"linear avg={row['linear_time_avg_s']*1e6:.1f}us  "
93                f"bin(rec) avg={row['binary_rec_time_avg_s']*1e6:.2f}us  "
94                f"hash avg={row['hash_time_avg_s']*1e6:.2f}us", flush=True)
95
96      with open(os.path.join(HERE, "results.json"), "w") as f:
97          json.dump(results, f, indent=2)
98      with open(os.path.join(HERE, "results.csv"), "w", newline="") as f:
99          w = csv.DictWriter(f, fieldnames=list(results[0].keys()))
100         w.writeheader(); w.writerows(results)
```

Figure 8.6 run_experiments.py, page 2 of 3

run_experiments.py (page 3/3)

```
101     return results
102
103
104 if __name__ == "__main__":
105     run()
106
```

Figure 8.7 run_experiments.py, page 3 of 3

extra_experiments.py

```
"""Two supporting experiments: (A) hash worst case, (B) hash build with GC off."""
import gc, random, time
import search_algorithms as sa

# A. adversarial keys: every key is a multiple of m -> all land in bucket 0
n = 10_000
m = sa.HashTable._next_prime(int(1.5 * n) + 1)
bad = [m * j for j in range(1, n + 1)]
ht = sa.HashTable(bad)
found, c = ht.search_counted(bad[-1])
print(f"A. adversarial keys, n={n:,}, m={m:,}: longest chain={max(len(b) for b in ht.table if b):,}, "
      f"comparisons for last key={c:,}")
t0 = time.perf_counter()
for _ in range(200): ht.search(bad[-1])
print(f"   time per search = {(time.perf_counter()-t0)/200*1e6:.1f} us (random keys: about 0.12 us)")

# B. hash build at n = 1,000,000 with and without the garbage collector
data = random.Random(2026 + 1_000_000).sample(range(190_000_000, 195_000_000), 1_000_000)
for label, off in (("GC on ", False), ("GC off", True)):
    gc.collect()
    if off: gc.disable()
    t0 = time.perf_counter(); sa.HashTable(data); t = time.perf_counter() - t0
    gc.enable()
    print(f"B. hash build n=1,000,000, {label}: {t:.3f} s")

```

Figure 8.8 extra_experiments.py

How to reproduce: python3 run_experiments.py (about one minute), then python3 extra_experiments.py and python3 make_figures.py. Absolute times depend on the machine, so they will differ slightly; the growth pattern should not.

Task 9 - Record Experimental Performance (3 Marks)

All values are measured by run_experiments.py (raw data in results.csv). Times are per search in microseconds unless stated.

Table 9.1 Average search time (microseconds), random present keys

n	Linear	Binary (recursive)	Binary (iterative)	Hash
1,000	9.76	1.07	0.87	0.12
10,000	110.7	1.57	1.19	0.12
100,000	1,575.1	3.15	2.50	0.19
1,000,000	18,846.3	2.45	1.86	0.12

Table 9.2 Worst-case search time (microseconds), absent key

n	Linear	Binary (recursive)	Binary (iterative)	Hash
1,000	20.5	0.91	0.62	0.12
10,000	212.0	1.28	0.98	0.10
100,000	3,258.1	2.72	2.19	0.11
1,000,000	24,786.5	1.90	1.38	0.11

Table 9.3 Key comparisons per search

n	Linear avg	Linear worst	Binary avg	Binary worst	Hash avg	Hash max seen
1,000	489.2	1,000	9.10	10	1.32	3
10,000	5,046.7	10,000	12.42	13	1.31	3
100,000	49,201.8	100,000	15.73	17	1.28	2
1,000,000	629,546.5	1,000,000	19.67	20	1.27	2

Table 9.4 Preprocessing cost and hash table statistics

n	Sort time (s)	Hash build (s)	Table size m	Longest chain	Mean chain (non-empty)
1,000	0.0001	0.0001	1,511	4	1.35
10,000	0.0015	0.0014	15,013	6	1.36
100,000	0.0219	0.0210	150,001	6	1.36
1,000,000	0.2919	1.1768	1,500,007	4	1.28

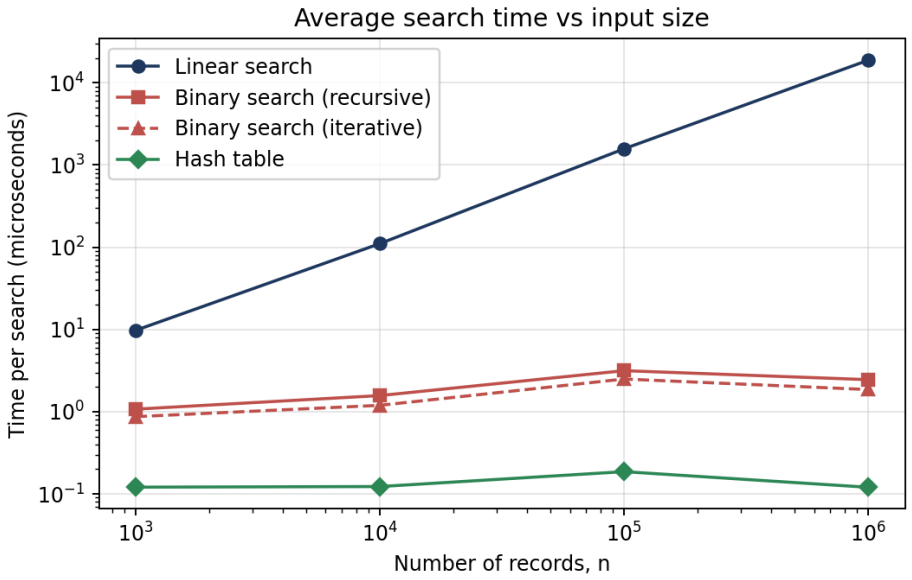


Figure 9.1 Average search time versus n (log-log axes)

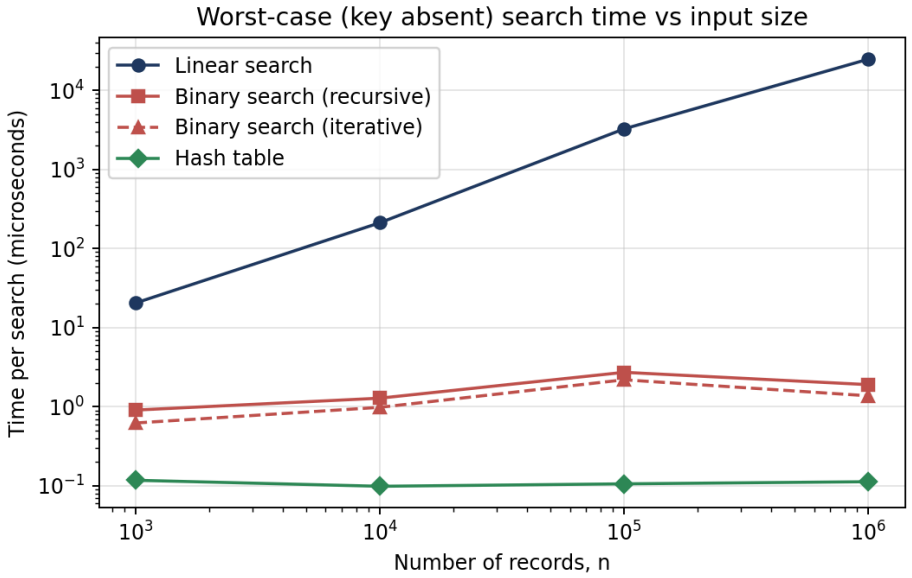


Figure 9.2 Worst-case (absent key) search time versus n

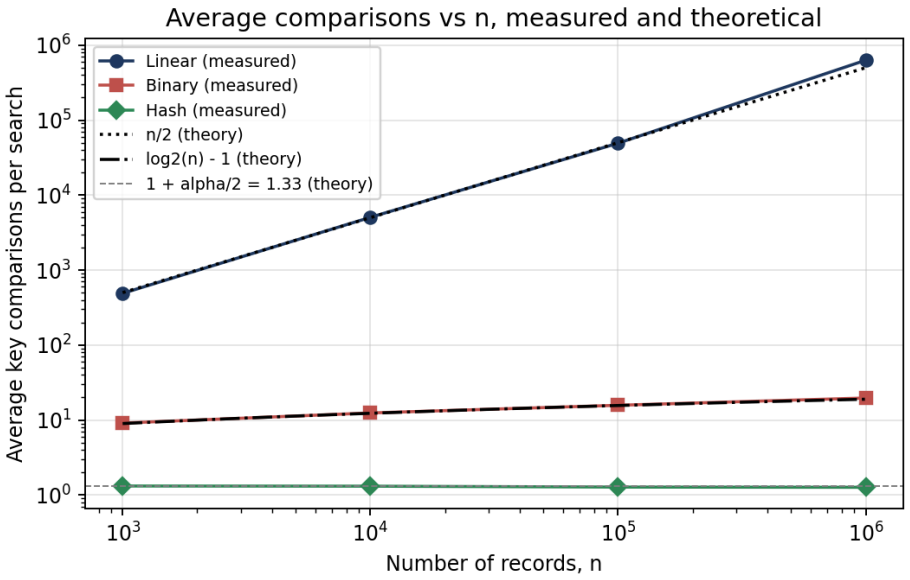


Figure 9.3 Average comparisons versus n, with theoretical curves n/2, log2(n) - 1 and 1.33

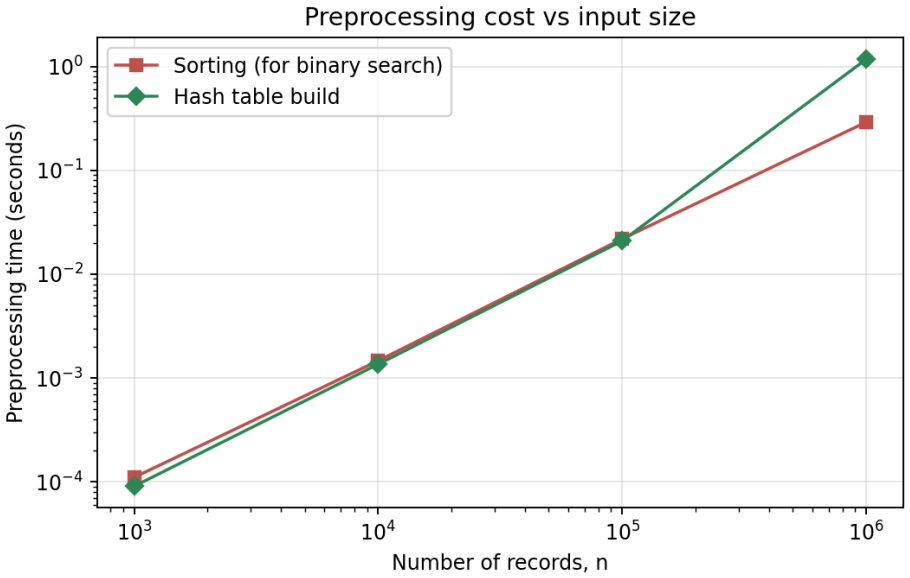


Figure 9.4 Preprocessing time versus n

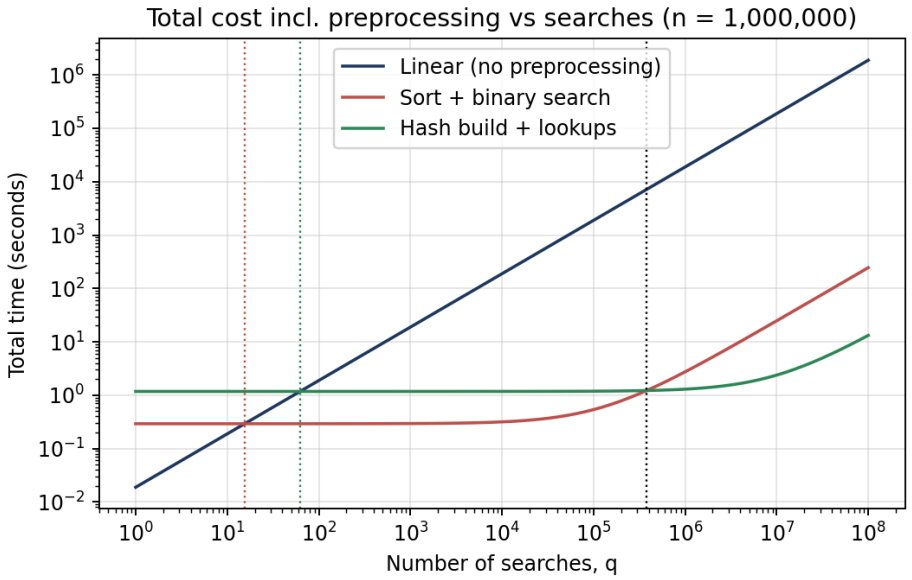


Figure 9.5 Total cost including preprocessing versus number of searches, n = 1,000,000 (dotted lines: break-even points)

Task 10 - Compare Theory with Experiments (4 Marks)

Operation counts agree closely with theory

Table 10.1 Measured / predicted average comparisons

n	Linear: measured / n/2	Binary: measured / (log2 n - 1)	Hash: measured / (1 + alpha/2)
1,000	489 / 500	9.10 / 8.97	1.32 / 1.33
10,000	5,047 / 5,000	12.42 / 12.29	1.31 / 1.33
100,000	49,202 / 50,000	15.73 / 15.61	1.28 / 1.33
1,000,000	629,547 / 500,000	19.67 / 18.93	1.27 / 1.33

Linear comparisons follow n/2 for the first three sizes. At n = 10^6 the measured mean is 26% above theory, but only 15 random keys were used; a uniform position over 10^6 records has a standard deviation of about 289,000, so the standard error of a 15-key mean is about 75,000 and the result is about 1.7 standard errors from n/2, which is ordinary sampling noise. The single exact comparison counts (middle record: 500,001) match theory exactly. Binary comparisons rise by about 3.3 for every tenfold increase in n, equal to log2 10 = 3.32. Hash lookups stay near 1.3 comparisons at every size, and the longest chain never exceeded 6 on the random data, so the uniform-hashing assumption holds for these keys.

Growth of running time

Linear search: for each tenfold increase in n the average time grew by factors of 11.3, 14.2 and 12.0 against a predicted 10, confirming Theta(n); the extra is cache and memory effects on larger lists. At n = 10^6 the absent-key scan takes 24.8 ms, about 25 ns per comparison in this interpreter. **Binary search:** time changes from 1.07 to 2.45 microseconds over a 1,000-fold increase in n, matching the near-doubling of probes (9.1 to 19.7). **Hash:** lookup stays at 0.12-0.19 microseconds throughout. Both are flat compared with the linear curve (Figure 9.1).

Preprocessing: sorting time grew by factors of 13.3, 14.9 and 13.3 per tenfold step, against predicted n log n ratios of 13.3, 12.5 and 12.0, confirming Theta(n log n). Hash build grew by 15.0, 15.4 and then 56.1, against a theoretical 10, so it is clearly worse than linear in practice at the largest size.

Which algorithm scales better

By search time: hash table > binary search >> linear search. At n = 10^6 a hash lookup takes 0.12 microseconds, a binary search 2.45 microseconds (about 20 times slower) and a linear scan 18.8 ms (about 7,700 times slower than binary search). Including preprocessing, the ranking depends on the number of searches, as the break-even analysis shows.

Table 10.2 Measured break-even points at n = 1,000,000

Comparison	Break-even (searches)	Meaning
Sort + binary vs linear	about 16	Sorting is repaid after 16 queries (theory by operation count: about 40)
Hash build vs linear	about 63	Hashing is repaid after 63 queries
Hash build vs sort + binary	about 379,800	Hashing wins overall only after roughly 380,000 searches per build

Why measurements differ from theory

- **Sampling noise in the linear average at 10^6.** Discussed above; the exact counts (first, middle, last, absent) match theory.

- **Binary search time is not exactly logarithmic.** It is 3.15 microseconds at 10^5 but 2.45 at 10^6. The 10^6 case timed only 15 distinct keys, each repeated 200 times, so the same few memory locations stay in the CPU cache; the 10^5 case used more distinct keys. Microsecond timings are also sensitive to timer resolution and system noise. The comparison counts are exact and confirm log n growth.
- **Hash build is slower than its Theta(n) count suggests.** The extra experiment (Figure 4.1) builds the 10^6 table in 1.17 s with Python's garbage collector on and 0.47 s with it off. Creating about a million small lists makes the collector run repeatedly, and the 1.5-million-slot table also no longer fits in cache. This is an implementation and memory-hierarchy effect, not a change of complexity class.
- **Sorting is fast in practice.** Python's built-in sorted() runs in compiled code, whereas the hash build and the searches are interpreted Python. That is why sorting (0.29 s) beats the hash build (1.18 s) at 10^6 even though n log n is larger than n, and why the measured sort break-even (16) is below the operation-count prediction (40).
- **Recursion costs a constant factor.** Recursive binary search is 1.2-1.4 times slower than the iterative loop (for example 2.45 against 1.86 microseconds at 10^6) because each call creates a stack frame. The asymptotic class is the same.
- **Machine dependence.** Absolute times depend on the computer and interpreter; only growth trends and ratios should be compared with theory.

Task 11 - Recommend and Defend the Final Design (4 Marks)

Recommendation for the original scenario: binary search on a sorted array of registration numbers.

The original scenario fixes no search rate and describes a database that grows. The design must therefore behave well for both few and many queries, be simple to maintain, and be predictable in the worst case.

Table 11.1 Evidence for the recommendation

Criterion	Linear	Binary + sort	Hash
Worst-case search (theory)	Theta(n)	Theta(log n)	Theta(n); expected Theta(1)
Average search at n = 10^6 (measured)	18.8 ms	2.45 us	0.12 us
Preprocessing at n = 10^6 (measured)	none	0.29 s	1.18 s
Searches to repay preprocessing vs linear	-	about 16	about 63
Memory	n slots	n slots	about 1.5 n slots plus chain overhead
Guaranteed worst case	yes (n)	yes (log n)	no (n possible; shown at 10,000 comparisons)
Ordered output / range queries	no	yes	no

- **Mathematical efficiency:** Theta(log n) per search after Theta(n log n) preprocessing; recursion depth at most 20 for a million records.
- **Experimental performance:** 2.45 microseconds per search at 10^6 records, about 7,700 times faster than linear search, with 19-20 probes in every case.
- **Preprocessing cost:** 0.29 s to sort 10^6 records, repaid after about 16 searches. It is the cheaper of the two preprocessing options (the hash build took four times longer in this implementation).
- **Against the hash table:** binary search is slower per lookup, but its preprocessing is cheaper, it uses less memory, its worst case does not depend on the quality of a hash function, and it supports ordered operations such as listing registration numbers in a range.
- **Against linear search:** beyond the first dozen or so queries, linear search is never competitive.
- **Scalability:** at 10^9 records binary search needs only about 30 probes, while linear search, extrapolating the measured 19 ms at 10^6, would need roughly 19 seconds per query.
- **Limitations:** the array must stay sorted, so inserting one record into the middle costs Theta(n) unless records are added in batches and merged in Theta(n + b). Hash insertion is cheaper (expected O(1)), and a hash table is better when lookups are very frequent (Task 12). The recommendation is therefore conditional on lookups not being so heavy that the extra build cost of hashing is repaid, which Task 12 tests.

Task 12 - Day-4 Design Challenge (8 Marks)

New requirements: 1,000,000 records; 50,000 search requests per hour (13.9 per second, 1,200,000 per day); new records added only once per day. **Decision: redesign around a hash index rebuilt once per day,** keeping the sorted array as a secondary structure for ordered queries. Measured values are those of Task 9 (n = 10^6).

A. Impact analysis (2 marks)

- **What changed:** the problem was one-off lookups on a growing database with no stated rate; it is now a sustained, known load of 1.2 million lookups per day on data that changes once per day.
- **Assumptions affected:** (1) "the cost of one search matters" becomes "the total cost of 1.2 million searches matters"; (2) "preprocessing is an overhead to avoid" becomes "preprocessing is paid once per day and shared by 1.2 million searches"; (3) "data keeps changing, so keep it easy to update" becomes "data is static for 24 hours".
- **Why search frequency matters:** per-search cost is multiplied by 50,000 every hour. At 18.8 ms per search, linear search would keep a processor busy for 942 s of every 3,600 s (26% of a core), with no headroom for bursts.
- **Why rare updates matter:** between updates the data is static, so any preprocessing is done once and amortised over the day: 0.29 s of sorting or 1.18 s of hash building spread over 1,200,000 searches is 0.24 and 0.98 microseconds per search. The usual weakness of sorted arrays and hash tables, expensive updates, stops mattering.

B. Evaluation of the original design (2 marks)

Table 12.1 Daily workload on 1,000,000 records (measured per-search times x 1.2 million searches)

Measure	Linear	Binary + sort (original)	Hash table
Time per search	18.8 ms	2.45 us	0.12 us
CPU time per hour of traffic	942 s (26% of a core)	0.12 s (0.003%)	0.006 s (0.0002%)
Daily preprocessing	none	0.29 s	1.18 s
Total CPU time per day	22,616 s (6.3 hours)	3.23 s	1.32 s
Basic operations per day (theory)	about 6 x 10^11	about 4.3 x 10^7	about 4.1 x 10^6

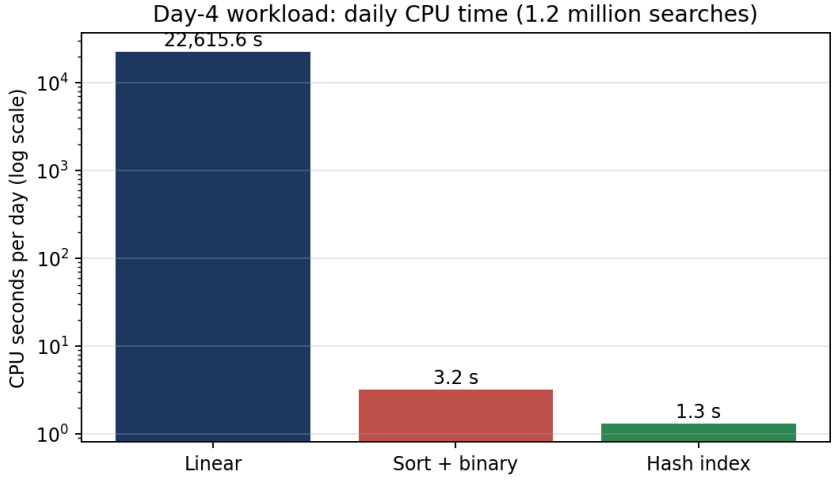


Figure 12.1 Daily CPU time for the three designs (log scale)

Expected behaviour of the original design. Binary search easily sustains the load: 0.12 s of processor time per hour is no bottleneck and the 0.29 s sort is done once a day. The original design is therefore adequate, but the hash table does better overall: 1.32 s per day against 3.23 s (about 2.4 times less), with about ten times fewer basic operations in theory. Linear search is clearly unsuitable (6.3 hours of CPU per day).

An honest note on the cross-over. Because the hash build is dearer than the sort here, sort-plus-binary is cheaper when fewer than about 380,000 searches are served per build (Figure 9.5). At 50,000 searches per hour that point is reached after about 7.6 hours, so over a full day (1.2 million searches) the hash table is the cheaper design, but for a build serving less than a third of a day the sorted array would win.

C. Adaptation or redesign (2 marks)

- **Revised strategy:** once per day, after the new records are loaded, build a hash table (h(K) = K mod m, m prime near 1.5n, separate chaining) over all records. All 50,000 hourly searches use this table. The sorted array is kept as a secondary structure, updated by merging the day's sorted new records into the old array in Theta(n + b) instead of re-sorting, and used for ordered or range queries and as a fallback.
- **What changed:** the primary search moves from comparison-based halving (Theta(log n)) to address calculation (Theta(1) expected); preprocessing moves from Theta(n log n) to Theta(n); about 50% more memory is used for the table.
- **Effect on complexity:** per-search cost falls from Theta(log n) (about 19 comparisons) to Theta(1) expected (about 1.3). Daily cost falls from Theta(n log n + 1.2 x 10^6 log n) to Theta(n + 1.2 x 10^6). The worst case is Theta(n) only if the hash function is defeated; a prime table size and the measured longest chain of 4-6 make that unlikely for registration numbers.
- **Retained from the original design:** the sorted array and binary search remain the right choice if the search rate were low or the data changed continuously.

D. Evidence-based defence (2 marks)

- **Mathematical analysis:** Theta(n + q) for hashing against Theta(n log n + q log n) for the original design. With n = 10^6 and q = 1.2 x 10^6 that is about 4.1 x 10^6 against about 4.3 x 10^7 basic operations.
- **Experiment:** 0.12 against 2.45 microseconds per lookup (about 20 times faster); about 1.3 against 19.7 comparisons; daily CPU 1.32 s against 3.23 s.
- **Search frequency:** the load is high enough to amortise the 1.18 s build, and it leaves large headroom: at 100 times the load (5 million requests per hour) the hash table would use about 0.6 s of CPU per hour, against 12 s for binary search.
- **Preprocessing:** 1.18 s once per day is trivial next to 86,400 s and fits the nightly update window, since records are added only once a day.
- **Trade-offs and honesty:** the hash table costs about 50% more memory, is unordered and has a theoretical Theta(n) worst case, which Figure 4.1 shows is real for adversarial keys. The experiment also shows that binary search would be sufficient for this workload (0.12 s of CPU per hour). The redesign is justified by lower total cost per day, headroom and constant-time scaling as the database grows, not because the original design fails. If ordered queries were required, the retained sorted array would serve them.

Summary and Conclusion

Table S.1 Key findings

Question	Answer
Fastest search at 10^6 records	Hash table, 0.12 us (binary 2.45 us, linear 18.8 ms)
Cheapest preprocessing	Sorting 0.29 s (hash build 1.18 s); linear needs none
Recommended design, original scenario	Binary search on a sorted array (Task 11)
Day-4 decision	Redesign: hash index rebuilt once a day, sorted array kept for ordered queries (Task 12)
Theory versus experiment	Operation counts match theory; timing differences are explained by caching, interpreter overhead, garbage collection and compiled sorting

Appendix - Reproduction notes

- All numbers in this report were produced by the code shown in Task 8 (python3 run_experiments.py, python3 extra_experiments.py, then python3 make_figures.py).
- The source-code and terminal "screenshots" are images rendered from the real source files and the real console output; they are not captures of a physical screen. Replace them with real screenshots from your own computer if your evaluator requires that.
- Timings depend on the machine and Python version; operation counts do not. A re-run will give slightly different times with the same growth pattern.